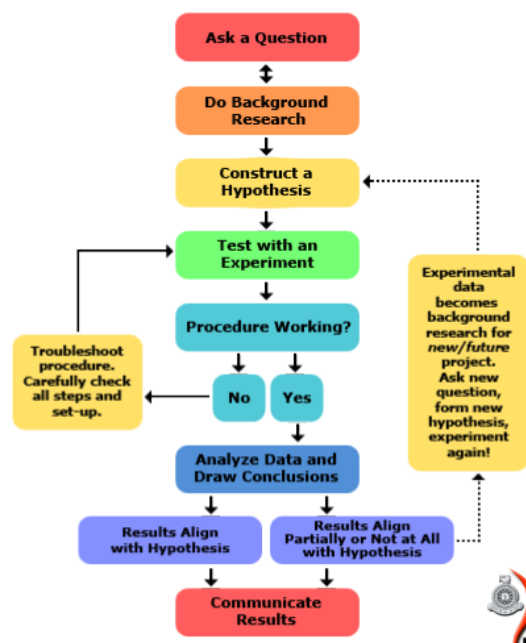


01. Introduction to Digital Forensics

- Digital Forensics is the science of identifying, preserving, analysing, and presenting digital evidence in a way that is legally admissible in court.
- Old incidents & crimes: physical evidence (fingerprints, bloodstains, eyewitness accounts)
- Modern incidents & crimes: Digital evidence (DNA analysis, biometrics, CCTV footage, facial recognition, data from computers/devices)
- Digital evidence: Any data stored or transmitted using a computer that support or refute a theory of how an offense occurred or that address critical elements of the offense.
- Digital evidence sources:
 - Open computer systems (laptops, desktops, smartphones, their storage devices), Embedded computer systems (smart wearables, printers, access control systems), Communication systems
- Forensic Readiness→evidence maximization, cost minimization, admissibility.
- Forensic Soundness→focuses on the process. Even if the data changes(live memory forensics), is the process documented, justified, and performed by a competent person? [If asked about acquiring RAM, you must admit it violates ACPO Principle 1 (it changes data) but is "forensically sound" because the volatile evidence would otherwise be lost.]
- Evidence Integrity→focuses on the data. md5 hash to check if evidence is identical from the moment of seizure to courtroom.
- Chain of Custody: A documented record showing,
 - Who handled the evidence, When they handled it, Where the transfer occurred, What actions they took, Why they transferred it
- w/o proper chain of custody, evidence can be inadmissible in court.
- Forensics is a science because it is testable and repeatable.
- 'Science' in digital forensics
 - DF is a scientific discipline, so it must follow the scientific method:
 - Ask question→Do background research→Construct hypothesis→Test with experiments→Check if procedure works→Analyse and draw conclusions→Communicate results
 - Key principles: Repeatability(same result using the same tool) & reproducibility(different investigator using a different tool gets the same result), Elimination of hypotheses
 - If a tool has a "black box" nature where the investigator doesn't know how it works, the evidence can be challenged because the "science" behind it cannot be explained.
- ACPO guidelines: Explains how to handle a digital crime scene
 - Best practices from the Association of Chief Police Officers
 - Not strict rules (helpful guidelines). May have to deviate under certain conditions. primarily designed for static forensics.



Principle 1: No action taken by law enforcement agencies or their agents should change data held on a computer or storage media which may subsequently be relied upon in court. [Violated if you plug a USB drive into a Windows machine without a hardware write-blocker (Windows writes "System Volume Information" automatically).]

Principle 2: In circumstances where a person finds it necessary to access original data held on a computer or on storage media, that person must be competent to do so and be able to give evidence explaining the relevance and the implications of their actions. [If a regular police officer (not a forensic specialist) starts clicking through files on a suspect's live laptop, they violate this because they cannot explain the technical implications of their actions in court.]

Principle 3: An audit trail or other record of all processes applied to computer-based electronic evidence should be created and preserved. An independent third party should be able to examine those processes and achieve the same result. [If you "find" a hidden file but didn't log the exact tool and command used, a third party cannot reproduce your results. The evidence becomes inadmissible.]

Principle 4: The person in charge of the investigation (the case officer) has overall responsibility for ensuring that the law and these principles are adhered to.

- Challenges to digital forensics:
 - Digital evidence is,
 - Complicated and messy
 - Generally an abstraction of some digital object or event. Mostly not direct data
 - Usually circumstantial, making it difficult to attribute computer activity to an individual
 - Can be manipulated or destroyed
- Data "Slacks" (Hidden Data)
 - When you save a small file, the computer might allocate a large "block" of space. The empty space at the end of that block is called Slack Space. Criminals hide data in Slack Space because it doesn't show up in normal folders.
- "Attribution." You can prove a message was sent from a laptop, but can you prove which person was sitting at the keyboard? That is the "circumstantial" nature of digital evidence

02. Data Representation in Storage Media

- In DF investigations, we mostly deal with mass storage devices. (FS forensics)
- Number systems: (Humans use decimal, computers use binary, forensic analysis use hexadecimal)
 - ASCII: a standard used to represent text characters in computer storage.
- Little Endian (Opposite way (Right to left, 2 byte blocks)) and Big Endian(Normal way (Left to right)): formats to store bytes in computer storage.
- Hexdump: Convenient way to view data. original binary data→turned into a table-like structure & displayed in hex format.

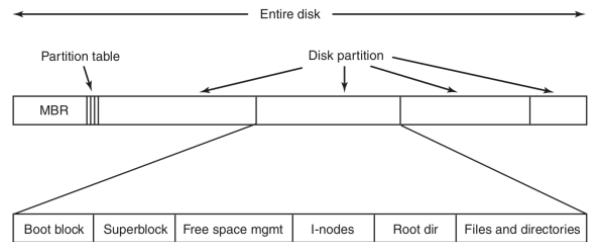
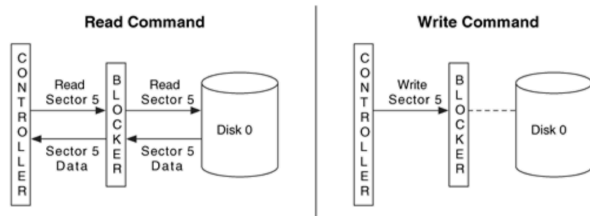
ASCII Chart

The following chart contains all 128 ASCII decimal (**dec**), octal (**oct**), hexadecimal (**hex**) and character (**ch**) codes.

dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch
0	0	00	NUL (null)	32	40	20	(space)	64	100	40	@	96	140	60	`
1	1	01	SOH (start of header)	33	41	21	!	65	101	41	A	97	141	61	a
2	2	02	STX (start of text)	34	42	22	"	66	102	42	B	98	142	62	b
3	3	03	ETX (end of text)	35	43	23	#	67	103	43	C	99	143	63	c
4	4	04	EOT (end of transmission)	36	44	24	\$	68	104	44	D	100	144	64	d
5	5	05	ENQ (enquiry)	37	45	25	%	69	105	45	E	101	145	65	e
6	6	06	ACK (acknowledge)	38	46	26	&	70	106	46	F	102	146	66	f
7	7	07	BEL (bell)	39	47	27	'	71	107	47	G	103	147	67	g
8	10	08	BS (backspace)	40	50	28	(	72	110	48	H	104	150	68	h
9	11	09	HT (horizontal tab)	41	51	29	)	73	111	49	I	105	151	69	i
10	12	0a	LF (line feed - new line)	42	52	2a	*	74	112	4a	J	106	152	6a	j
11	13	0b	VT (vertical tab)	43	53	2b	+	75	113	4b	K	107	153	6b	k
12	14	0c	FF (form feed - new page)	44	54	2c	,	76	114	4c	L	108	154	6c	l
13	15	0d	CR (carriage return)	45	55	2d	-	77	115	4d	M	109	155	6d	m
14	16	0e	SO (shift out)	46	56	2e	.	78	116	4e	N	110	156	6e	n
15	17	0f	SI (shift in)	47	57	2f	/	79	117	4f	O	111	157	6f	o
16	20	10	DLE (data link escape)	48	60	30	0	80	120	50	P	112	160	70	p
17	21	11	DC1 (device control 1)	49	61	31	1	81	121	51	Q	113	161	71	q
18	22	12	DC2 (device control 2)	50	62	32	2	82	122	52	R	114	162	72	r
19	23	13	DC3 (device control 3)	51	63	33	3	83	123	53	S	115	163	73	s
20	24	14	DC4 (device control 4)	52	64	34	4	84	124	54	T	116	164	74	t
21	25	15	NAK (negative acknowledge)	53	65	35	5	85	125	55	U	117	165	75	u
22	26	16	SYN (synchronous idle)	54	66	36	6	86	126	56	V	118	166	76	v
23	27	17	ETB (end of transmission block)	55	67	37	7	87	127	57	W	119	167	77	w
24	30	18	CAN (cancel)	56	70	38	8	88	130	58	X	120	170	78	x
25	31	19	EM (end of medium)	57	71	39	9	89	131	59	Y	121	171	79	y
26	32	1a	SUB (substitute)	58	72	3a	:	90	132	5a	Z	122	172	7a	z
27	33	1b	ESC (escape)	59	73	3b	;	91	133	5b	[	123	173	7b	{
28	34	1c	FS (file separator)	60	74	3c	<	92	134	5c	\	124	174	7c	
29	35	1d	GS (group separator)	61	75	3d	=	93	135	5d	]	125	175	7d	}
30	36	1e	RS (record separator)	62	76	3e	>	94	136	5e	^	126	176	7e	~
31	37	1f	US (unit separator)	63	77	3f	?	95	137	5f	_	127	177	7f	DEL (delete)

03. Structure of Non-volatile Storage

- Forensic investigations need to locate forensic artefacts from storage devices.
- Forensic tools can make mistakes when extracting those artefacts, or malicious users can take actions to fool the forensic tools (anti-forensics). So, manual inspection can be done.
- Analysing a forensic disk image→risky. A copy has to be used for inspection, but has to be guaranteed it's an exact copy.
- Tools to take disk images: dd command, FTK Imager
- Using dd command:
 - Step 1: Boot computer from a live Linux CD or USB stick (using the original OS is risky. Might get modified)
 - Step 2: md5sum /dev/sda > ~/Desktop/hash-value.md5 (calculate hash of original disk)
 - Step 3: dd if=/dev/sda of=~/Desktop/disk-image.dd bs=1 (copy entire disk, 1 block at a time, to create the disk image)
 - Step 4: md5sum ~/Desktop/disk-image.dd >> ~/Desktop/hash-value.md5 (calculate hash of copy)
 - Step 5: md5sum /dev/sda > ~/Desktop/hash-value.md5 (calculate hash of original again)
- Write blockers help prevent accidental or malicious modifications of a storage device.
 - They're hardware devices that allow READ operations (to create an image), and block WRITE operations (to prevent accidental modifications)
- Disk address example -> Cylinder 100, Track 5, Sector 42



Master Boot Record

- The 1st 512 bytes of a disk that is organised using DOS partitions.
- Contains boot code (446 bytes), partition table with 4 entries (16 bytes each), and signature value (2 bytes)
- Boot code contains the instructions that tell the computer how to process the partition table and locate the OS.
 - Contains small bootloader code, and is executed immediately after the BIOS hands over control.
 - Reads the partition table, finds the active (bootable) partition, and loads the bootloader from that partition.
- The partition table has 4 entries and each describes a DOS partition. (boot flag, partition type, starting sector, partition size)
- Boot signature: fixed value (0x55AA), used by BIOS to verify that the sector is a valid boot sector. If missing, the disk is non-bootable.

Master Boot Record and Partition Table Data Structures

The Master Boot Record (MBR) occupies the first sector on a disk. Table 1 shows the structure of the MBR. The Extended Boot Record (EBR) has the same structure, except that the bootstrap code is empty.

Offset (Hex)	Length (Dec)	Description
0x00	446	The Master Boot Record. This area contains boot strap code for the computer system.
0x1BE	64	Partition Table – contains information on four partitions. Each partition requires 16 bytes of storage space
0x1FE	2	Signature [0x55AA]

Table 1: Master Boot Record Structure

Offset	Size (bytes)	Description
0x000	440 ¹	MBR Bootstrap (flat binary executable code)
0x1B8	4	Optional "Unique Disk ID / Signature" ²
0x1BC	2	Optional, reserved 0x0000 ³
0x1BE	16	First partition table entry
0x1CE	16	Second partition table entry
0x1DE	16	Third partition table entry
0x1EE	16	Fourth partition table entry
0x1FE	2	(0x55, 0xAA) "Valid bootsector" signature bytes

Offset ¹	Size (bytes)	Description
0x00	1	Drive attributes (bit 7 set = active or bootable)
0x01	3	CHS Address of partition start
0x04	1	Partition type
0x05	3	CHS address of last partition sector
0x08	4	LBA of partition start
0x0C	4	Number of sectors in partition

Table 2 shows the partition table structure and table 3 shows a selection of partition types. A more complete list of partition types is available at: http://en.wikipedia.org/wiki/Partition_type

Offset (Hex)	Length (Dec)	Description	Identifier	Type
0x00	1	Partition type: 0x80 – Bootable 0x00 – Non Bootable	0x05	Extended
0x01	3	Start Position CHS	0x06	FAT 16
0x04	1	Partition Type (see table 3)	0x07	NTFS
0x05	3	End Position CHS	0x0B	FAT 32
0x08	4	Start Position LBA	0x82	Linux Swap
0x0C	4	Number of Sectors	0x83	Linux (Ext)

Table 3: Partition Type Identifiers

Table 2: Partition Table Entry Structure

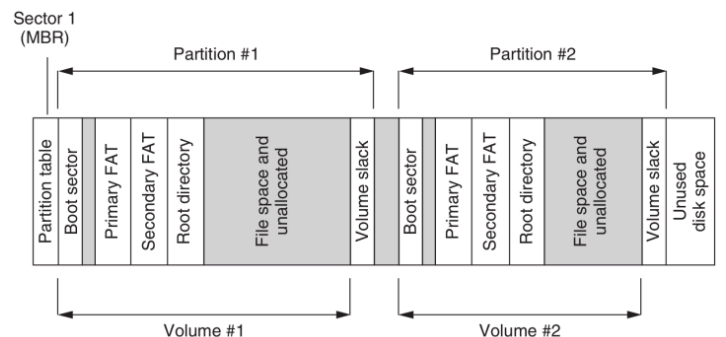
```
# Extracting the partition table
dd if=MBR_WS.dd of=partition-table.dd bs=1 skip=446 count=64

# Interpreting the partition table
## Entry 1
0000 0101 073f 2010 0008 0000 0080 0000
Bootable? No
What file system? 07 -> NTFS
Where is it? 00 08 00 00 -> LE to BE -> 00 00 08 00 -> Hex to Dec -> 2048 sectors
How big it is? 00 80 00 00 -> LE to BE -> 00 00 80 00 -> Hex to Dec -> 32768 sectors
```

- Extended Boot Record (EBR) has the same structure, except that the bootstrap code is empty.
 - In its first sector, there is the MBR with the partition table and the bootloader code.
 - One of the 4 primary partitions is naked as extended. Then it would have 4 partitions. $(448 + (16 \cdot 4) + 2)$
 - To extract: `dd if=MBR_WS.dd of=extended-partition.dd bs=512 skip=100352 count=262144`
- The bootloader (like GRUB or Windows Boot Manager): Displays the OS selection menu and loads the selected OS.
- File Systems help to logically organise the data in the storage.
 - Usually organised in a tree-like structure using directories and files.
 - A file or directory has data (actual content) and metadata (info about the file)
- Office file cabinet analogy:
 - In a physical file storage in an office, there are lots of files stored in cabinets. To ensure that the files can be located easily, a catalogue can be used.
 - This is what a FS does. They are special data structures that do the duty of the catalogue.
- A file system can be manipulated with multiple operations.
 - Most OSs implement the POSIX interface to interact with file systems.
 - Create a file -> `mkdir`, `touch`, `fopen()`
 - Open a file -> `fopen()`, `open()`
 - Writing to a file -> `fwrite()`, `write()`
 - Reading from a file -> `fread()`, `read()`
 - Closing a file -> `fclose()`, `close()`
 - Deleting a file -> `rm`, `unlink()`
 - Repositioning -> `seek()`
 - Almost all high level programming languages provide this POSIX interface to the file system.
- FAT: used by SD cards, USB drives and older Windows systems.
 - Simple structure, easy to understand
 - Limited file size support
- NTFS: used by Modern Windows
 - Has journaling, encryption, access control
 - Complex structure with many hidden files
- EXT: used by Linux (modern version - EXT4)
 - Journaling support (good for recovery)

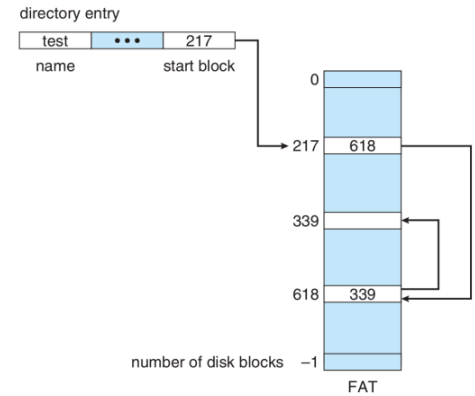
04. FAT File System

- Default file system for Microsoft DOS and Windows 9x
- Widely used on removable drives like SD cards and USB sticks
- Supported by almost all modern OS.
- There are 3 versions: FAT12, FAT16, FAT32 (depends on the size of data structures)



OS

- Basic functionality of FAT:
 - Step 1: Look up the file in directory
 - File 'test.txt' starts at cluster 217 in data area
 - Step 2: Follow chain in the FAT
 - FAT is a linked list. Each entry tells which cluster to go next
 - 217 -> 618 -> 339 -> end of file marker
 - Step 3: Read the file by the chain → Complete file= clust 217 + clust 618 + clust 339
- Why this design?: because a large file might not fit in one cluster (usually clusters are 4KB-64KB). So the file is split across multiple clusters.
- Sector: Physical unit on disk (usually 512 bytes)
- Cluster: Logical unit used by file system



05. FAT File System Analysis

- Boot sector: 1st 512 bytes of a FAT partition. Contains critical info about the FAT FS.

FAT File System Data Structures

This document contains the tables required to interpret hex dumps of FAT file system structures. All data in these tables is stored in little-endian format unless otherwise stated. The tables include:

- Table 1: FAT Boot Sector Structure
- Table 2: FAT Root Directory Entry Structure
- Table 3: FAT Root Directory Attribute Values

Table 1: FAT Boot Sector Structure

Offset (Hex)	Length (Bytes)	Description
0x00	3	Address for boot-strapping code
0x03	8	OEM Name (mkdofs)
0x0B	2	The number of bytes in each sector
0x0D	1	The number of sectors in each cluster
0x0E	2	The number of reserved sectors
0x10	1	The number of FATs
0x11	2	The maximum number of files in the Root Directory – each entry is 32 bytes in size, hence this number * 32 gives the size of the root directory.
0x13	2	Total number of sectors (if two bytes are not large enough these bytes are zeroed and four bytes at 0x20 – 0x23 are used)
0x15	1	No longer required: Originally media type: F0 – Removable; F8 – Fixed. Now most removable media are USB keys which count as Hard Drives, and often appear as 0xF8 (Fixed media).
0x16	2	The number of sectors in each FAT
.....		

```
# Extracting Boot sector
dd if=fat16.dd of=boot-sector.dd bs=1 skip=0 count=512

# Analysing Boot sector

## Size of a sector: 00 02 -> LE to BE -> 02 00 -> Hex to Dec -> 512 bytes
## Size of a cluster: 04 -> Hex to Dec -> 4 sectors
## Number of reserved sectors: 04 00 -> LE to BE -> 00 04 -> Hex to Dec -> 4 sectors
## Number of FAT tables: 02 -> Hex to Dec -> 2 tables
## Size of a FAT table: 20 00 -> LE to BE -> 00 20 -> Hex to Dec -> 32 sectors
## Number of entries in the root directory: 00 02 -> LE to BE -> 02 00 -> Hex to Dec -> 512 entries
## Size of root directory in bytes: (512 x 32) bytes
## Size of root directory in sectors: (512 x 32) bytes / 512 bytes = 32 sectors
```

- FAT table: contains 2 byte pairs representing each cluster in the data area.
 - The 1st 4 bytes are reserved. So they are not considered.
 - After that, each pair represents a cluster and the cluster numbering starts with 2

```
00000000: f8ff ffff 0000 ffff ffff 0600 0700 0800 .....
00000010: 0900 0a00 0b00 0c00 0d00 0e00 0f00 1000 .....
00000020: 1100 1200 1300 1400 1500 1600 1700 ffff .....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000 .....
```

- Ex:
 - Initial 4 bytes are not considered. Next 2 bytes (0000) represent cluster 2. Next 2 bytes (ffff) -> cluster 3
 - Cluster 8 is represented by the 2 bytes with the value 0900
- 0x0000 -> unallocated cluster
- 0xFFFF7 -> Cluster contains a bad sector and cannot be used
- Between 0xFFFF8 and 0xFFFFF -> Cluster is allocated and it is the last cluster of the sequence of clusters allocated to a file
- Anything else -> Cluster is allocated and the value stored in the entry represents the address of the next cluster in the sequence

Region	Purpose	Content
Boot sector	System startup info	FS parameters, cluster size
Primary FAT	Cluster allocation	Chain showing which clusters belong to which file
Secondary FAT	Backup FAT	Identical copy of for recovery if Primary fails
Root Directory	File index	List of all root level files/folders
File Space	Data storage	Actual file data and deleted files

Root Directory

- Basically a large table with entries.
- Each entry represents a file or a directory that is directly under the root directory of the FS.
- Each directory entry is always fixed 32 bytes
- By interpreting the attributes in an entry, we can find out all required information to retrieve a file represented by that entry.
- Some entries -> represent a directory that is under the root directory.
 - Then, the details point to some data blocks in the FS where another similar directory data structure is stored.
 - Then that directory structure has to be retrieved and its entries should be interpreted to find the files under that directory.

Table 2: FAT Root Directory Entry Structure

Offset (Hex)	Length (Bytes)	Description
0x00	1	First character of the file name (ASCII) or allocation status: 0x00 unallocated; 0xE5 deleted.
0x01	10	Remaining characters of the filename. The '.' is implied between 0x07 and 0x08.
0x0B	1	File attributes (see table 3)
0x0C	1	Reserved for future use
0x0D	1	Creation Time (tenths of seconds)
0x0E	2	Creation Time
0x10	2	Creation Date
0x12	2	Access Date
0x14	2	High-order bytes of the first cluster location (always 0x0000 in FAT 12 / 16)
0x16	2	Modified Time
0x18	2	Modified Date
0x1A	2	Low order bytes of the first cluster location
0x1C	4	File Size

```
# Extracting the FAT table:
dd if=fat16.dd of=fat-table.dd bs=512 skip=4 count=32

# Extracting the Root Directory:

Offset: reserved area + FAT1 + FAT2 = 4 + 32 + 32 = 68 sectors
Size: size of root directory = 32 sectors

dd if=fat16.dd of=root-dir.dd bs=512 skip=68 count=32
```

Table 3: FAT Root Directory Attribute Values

Flag Value	Description
0b0000 0001	Read Only
0b0000 0010	Hidden File
0b0000 0100	System File
0b0000 1000	Volume Label
0b0000 1111	Long File Name
0b0001 0000	Directory
0b0010 0000	Archive

Structure for LFN Entries

Byte Range	Description
0	Sequence Number and allocation Status
1 – 10	File Name Characters (Unicode)
11	File Attributes
12	Reserved
13	Checksum
14 – 25	File Name Characters (Unicode)
26 – 27	Reserved
28 – 31	File Name Characters (Unicode)

Here, in the LFN entry, only the byte range from 1-10, 14-25, and 28-31 contains the LFN characters. So these bytes are taken and the Hex values are converted to Unicode to extract the long file name.

Entry 1 (from assignment):
LFN -> 6d 0065 0073 0073 0061 00 67
0065 002e 0074 0078 0074 -> Hex to
Unicode -> **message.txt**

```
# Interpreting the 1st entry in the Root Directory:
5445 5354 2020 2020 5458 5420 0000 077e
3943 3943 0000 077e 3943 0300 6800 0000

## File name: 5445 5354 2020 2020 5458 54 -> TEST   TXT -> TEST.TXT
## Is it deleted? Not deleted.
## Is it a folder or a file? 20 -> 2 0 -> 0010 0000 -> It is a file.
## File size: 68 00 00 00 -> LE to BE -> 00 00 00 68 -> Hex to Dec -> 104 bytes

## First data cluster allocated it:
- Higher order bytes: 00 00 -> LE to BE -> 00 00
- Lower order bytes: 03 00 -> LE to BE -> 00 03
- First cluster address: 00 00 00 03 -> Hex to Dec -> cluster 3

## Are there any other clusters allocated to this file?
    There are no other clusters allocated to this file.

## Extracting cluster 3 in the data area:
    Offset to cluster 3:
        reserved area + FAT1 + FAT2 + Root Dir + cluster 2 = 4 sectors + 32 sectors + 32 sectors +
        32 sectors + 4 sectors
        = 104 sectors

    Size of cluster 3 : 4 sectors

    dd if=fat16.dd of=cluster-3.dd bs=512 skip=104 count=4

## Extracting the actual file:

    dd if=cluster-3.dd of=TEST.TXT bs=1 skip=0 count=104
```

06. NTFS (New Technology File System)

- Default FS of all modern Windows OS.
- Highly scalable FS - supports huge files and partitions
- More features than FAT
- A jouralling FS - all operations are logged so that when something goes wrong, operations can be reverted.
- NTFS considers everything as a file. Even metadata are stored as special files.
- The 1st sector of an NTFS partition contains the boot sector, while the rest can contain file system metadata all over the place.

Metadata files

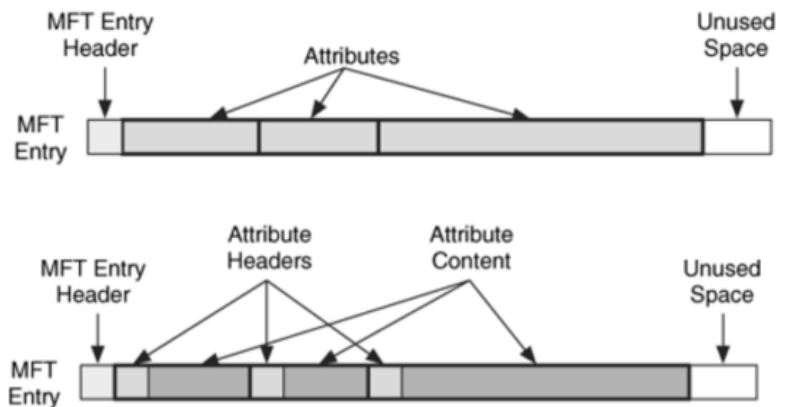
- There is a collection of metadata files in the NTFS
- NTFS partition structure:
- \$Boot - Boot sector of the FS
- \$MFT - Master File Table
- \$MFTMirr - a backup copy of the \$MFT
- \$LogFile - A journal file that records the transactions for recovery purposes
- . - Root directory of the FS
- \$Bitmap - The allocation status of the cluster in the FS

Table 1: Metadata Files

File Record #	Filename	Description
0	\$MFT	Index of every file on the volume
1	\$MFTMirr	Backup of \$MFT records
2	\$LogFile	Transactional Logging
3	\$Volume	Volume name, version, state of the volume
4	\$AttrDef	Attributes definition
5	.	Root directory of the volume
6	\$Bitmap	In use / Free cluster volume map.
7	\$Boot	NTFS Volume Boot Record
8	\$BadClus	Bad Clusters List

\$MFT

- There is an MFT entry for each file and directory in the \$MFT
- Each \$MFT entry is 1024 bytes, and has a header of 42 bytes. The rest of the entry has a number of attributes.
- Different \$MFT entries have different attributes depending on the requirements.
- Each attribute has its own header and body.
- The attribute header specifies the attribute type, size, and name.
- Resident attributes -> both header and body are located within the \$MFT record.
- Non-resident attributes -> only the header is in the \$MFT record. The rest is somewhere else.



Type Identifier	Name	Description
16	\$STANDARD_INFORMATION	General information, such as flags; the last accessed, written, and created times; and the owner and security ID.
32	\$ATTRIBUTE_LIST	List where other attributes for file can be found.
48	\$FILE_NAME	File name, in Unicode, and the last accessed, written, and created times.
64	\$VOLUME_VERSION	Volume information. Exists only in version 1.2 (Windows NT).
64	\$OBJECT_ID	A 16-byte unique identifier for the file or directory. Exists only in versions 3.0+ and after (Windows 2000+).
80	\$SECURITY_DESCRIPTOR	The access control and security properties of the file.
96	\$VOLUME_NAME	Volume name.
112	\$VOLUME_INFORMATION	File system version and other flags.

Type Identifier	Name	Description
128	\$DATA	File contents.
144	\$INDEX_ROOT	Root node of an index tree.
160	\$INDEX_ALLOCATION	Nodes of an index tree rooted in \$INDEX_ROOT attribute.
176	\$BITMAP	A bitmap for the \$MFT file and for indexes.
192	\$SYMBOLIC_LINK	Soft link information. Exists only in version 1.2 (Windows NT).
192	\$REPARSE_POINT	Contains data about a reparse point, which is used as a soft link in version 3.0+ (Windows 2000+).
208	\$EA_INFORMATION	Used for backward compatibility with OS/2 applications (HPFS).
224	\$EA	Used for backward compatibility with OS/2 applications (HPFS).
256	\$LOGGED_UTILITY_STREAM	Contains keys and information about encrypted attributes in version 3.0+ (Windows 2000+).

\$Boot

- The metadata file entry number 7 in the \$MFT table
- Contains critical info about the FS.
- Has the \$DATA attribute (non-resident)
 - The actual content of it is located in the first sector of the partition.

Table 2: \$Boot Structure

Offset (Hex)	Length (Bytes)	Description
0x00	3	Jump Instruction
0x03	8	OEM ID – ASCII values for the string “NTFS ”
0x0B	2	Number of bytes per sector
0x0D	1	Number of sectors per cluster
0x28	8	Total number of sectors in the volume
0x30	8	Starting cluster for \$MFT
0x38	8	Starting cluster for \$MFTMirr
0x40	1	Signed byte value indicating the fixed size of each \$MFT file record. If this number is negative, the fixed size is 2 to the power of the absolute value of the number. This is usually -10, which means the File record size is $2^{10} = 1024$ bytes.
0x48	8	NTFS Volume Serial Number
0x54	426	Bootstrap Code
0x01FE	2	End of sector signatute (0x55AA)

```

00000000: eb52 904e 5446 5320 2020 2000 0208 0000 .R.NTFS .....
00000010: 0000 0000 00f8 0000 2000 4000 0008 0000 .....@.....
00000020: 0000 0000 8000 8000 ffff 0100 0000 0000 .....
00000030: 0400 0000 0000 0000 ff1f 0000 0000 0000 .....
00000040: f600 0000 0100 0000 bb87 5b42 f4d9 a769 .....[B...i
00000050: 0000 0000 fa33 c08e d0bc 007c fb68 c007 .....3.....h..
00000060: 1f1e 6866 00cb 8816 0e00 6681 3e03 004e ...hf.....f>..N
00000070: 5446 5375 15b4 41bb aa55 cd13 720c 81fb TFSu..A..U..r...
00000080: 55aa 7506 f7c1 0100 7503 e9d2 001e 83ec U..u.....
00000090: 1868 1a00 b448 8a16 0e00 8bf4 161f cd13 ..h..H.....
000000a0: 9f83 c418 9e58 1f72 e13b 060b 0075 dba3 .....X.R;...u..
000000b0: 0f00 c12e 0f00 041e 5a33 dbb9 0020 2bc8 .....Z3....+.
000000c0: 66ff 0611 0003 160f 008e c2ff 0616 00e8 f.....
000000d0: 4000 2bc8 77ef b800 bbcd 1a66 23c0 752d @.+..w.....f#..u
000000e0: 6681 fb54 4350 4175 2481 f902 0172 1e16 f..TCPAu$....r..
000000f0: 6807 bb16 6870 0e16 6809 0066 5366 5366 h...hp..h..fSfSf
00000100: 5516 1616 6808 0166 610e 07cd 1ae9 6a01 U..h..fa....j..
00000110: 9090 6660 1e06 66a1 1100 6603 061c 001e ..f'.f...f....
00000120: 6668 0000 0000 0000 6650 0653 6801 0068 1000 fh....fP.Sh..h..
00000130: b442 8a16 0e00 161f 8bf4 cd13 6659 5b5a .B.....fY[Z
00000140: 6659 6659 1f0f 8216 0066 ff06 1100 0316 fYfY.....f.....
00000150: 0f00 8ec2 ff0e 1600 75bc 071f 6661 c3a0 .....u...fa..
00000160: f801 e808 00a0 fb01 e802 00eb feb4 018b .....
00000170: f0ac 3c00 7409 b40e bb07 00cd 10eb f2c3 ..<.t.....
00000180: 000a 4120 6469 736b 2072 6561 6420 6572 ..A disk read er
00000190: 726f 7220 6f63 6375 7272 6564 000d 0a42 ror occurred...B
000001a0: 4f4f 544d 4752 2069 7320 6d69 7373 696e 00TMGR is missin
000001b0: 6700 0d0a 424f 4f54 4d47 5220 6973 2063 g...BOOTMGR is c
000001c0: 616d 7072 6573 7365 6400 0d0a 6572 6573 ompressed...Pres
000001d0: 7320 4374 726c 2b41 6c74 2b44 656c 2074 s Ctrl+Alt+Del t
000001e0: 6f20 7265 7374 6172 740d 0a00 0000 0000 o restart.....
000001f0: 0000 0000 0000 0000 809d b2ca 0000 55aa .....U..

```

```
# Extracting $Boot file's content
```

```
dd if=ntfs.dd of=boot.dd bs=1 skip=0 count=512
```

```
-----
```

```
# Analysing the $Boot file
```

```
Size of a sector (in bytes): 00 02 -> LE to BE -> 200 -> Hex to Dec -> 512 bytes
```

```
Size of a cluster (in sectors): 08 -> LE to BE -> Hex to Dec -> 8 sectors
```

```
Size of the partition (in sectors): ff ff 01 00 00 00 00 00 -> LE to BE -> 01 ff ff -> Hex to Dec -> 131071 sectors -> 131071 x 512 bytes -> 67108352 bytes
```

```
Starting cluster of $MFT file: 0400 0000 0000 0000 -> LE to BE -> 04 -> Hex to Dec -> Cluster 4
```

```
# Extracting $MFT file's 0th entry
```

```
offset: 4 clusters -> 4x8 sectors -> 4x8x512 bytes -> 16384 bytes
size: 1024 bytes
```

```
dd if=ntfs.dd of=mft-entry-0.dd bs=1 skip=16384 count=1024
```

```
-----
```

```
# Extracting $MFT file's 1st entry
```

```
offset: offset to the 0th entry + size of 0th entry = 16384 + 1024 = 17408 bytes
size: 1024 bytes
```

```
dd if=ntfs.dd of=mft-entry-1.dd bs=1 skip=17408 count=1024
```

```
-----
```

```
# Extracting $MFT file's 2nd entry
```

```
offset: offset to the 0th entry + size of 0th entry + size of 1st entry = 16384 + (1024 x 2) = 18432 bytes
size: 1024 bytes
```

```
dd if=ntfs.dd of=mft-entry-2.dd bs=1 skip=18432 count=1024
```

```
-----
```

```
# Extracting $MFT file's 64th entry
```

```
offset: 16384 + (1024 x 64) = 81920 bytes
size: 1024 bytes
```

```
dd if=ntfs.dd of=mft-entry-64.dd bs=1 skip=81920 count=1024
```

- Every \$MFT entry can be extracted now:

- So the first thing in a \$MFT entry is its own header, which is 42 bytes long.

Table 3: File Record Header Structure

Offset (Hex)	Length (Bytes)	Description
0x00	4	Signature “FILE” or occassionally “BAAD”
0x08	8	\$Logfile Update Sequence Number
0x10	2	Sequence Count
0x12	2	Filenames count
0x14	2	Offset to the first attribute (also the header size)
0x16	2	Allocation Status (see table 4)
0x18	4	Logical Size of the file record
0x1C	4	Physical Size of the file record
0x28	2	Next Attribute ID (also the number of attributes)

Table 4: Allocation Status

Hex	Binary	File / Folder	Deleted / Allocated
0x00	0000 0000	File	Deleted
0x01	0000 0001	File	Allocated
0x02	0000 0010	Folder	Deleted
0x03	0000 0011	Folder	Allocated

- This is the 64th entry's hex view->

```
# Interpreting 64th $MFT entry's header

offset to the first attribute: 38 00 -> LE to BE -> 0x38 -> Hex to Dec -> 56 bytes

dd if=mft-entry-64.dd bs=1 skip=56 status=none | xxd

Size of the first attribute: 4800 0000 -> LE to BE -> Hex to Dec -> 72 bytes

## Extracting attribute 1

dd if=mft-entry-64.dd of=mft-64-attr-1.dd bs=1 skip=56 count=72

## Extracting attribute 2

offset: offset of first attribute + size of first attribute = 56 + 72 = 128 bytes

dd if=mft-entry-64.dd bs=1 skip=128 status=none | xxd

size: 7000 0000 -> LE to BE -> 70 -> Hex to Dec -> 112 bytes

dd if=mft-entry-64.dd of=mft-64-attr-2.dd bs=1 skip=128 count=112
```

Offset in Record	Size in Bytes	Description
0x00	4	Always: FILE (or Magic Number = 0x454C4946; or as they appear in a disk editor: "46 49 4C 45")
0x04	2	Offset to 'Update Sequence...Array' (from start of FILE header); often called the "Fix-up Array" in forensic circles. (Prior to NTFS v. 3.1, this was 0x002A. For NTFS v. 3.1, it became 0x0030.)
0x06	2	Size in WORDs of the 'Update Sequence Number and Array' (USA)
0x08	8	\$LogFile Sequence Number (or LSN)
0x10	2	Sequence Number
0x12	2	Hard Link Count
0x14	2	Offset to first Attribute
0x16	2	Flags
0x18	4	Actual Size of this FILE Record.
0x1C	4	Allocated Size of this FILE Record; normally 0x400 (1 KiB; 2 sectors).
0x20	8	File Reference to Base FILE Record
0x28	2	Next Attribute ID
0x2A	2	Used to align to 4-byte boundary (only since NTFS version 3.1)
0x2C	4	MFT Record Number (only since NTFS version 3.1)

Table 5: File Record Attribute Types

Type (0x)	Location	OS	Name	Description
1000 0000	Resident		\$Standard_Information	Traditional metadata: Timestamps; Flags; Access control; etc.
2000 0000	Non-Resident		\$Attribute_List	Attributes to be found outside this list
3000 0000	Resident		\$File_Name	Name of the file. A file may have multiple names
4000 0000	Resident	NT only	\$Volume_Version	NTFS Version
4000 0000	Resident	2K+	\$Object_ID	ID for the file record
5000 0000	Resident		\$Security_Descriptor	Security settings
6000 0000	Resident		\$Volume_Name	Name of volume, only used in \$VOLUME
7000 0000	Resident		\$Volume_Information	NTFS Version and dirty flag bit. Only used in \$VOLUME metadata.
8000 0000	Either		\$Data	The actual contents of the file.
9000 0000	Resident		\$Index_Root	Directory data used by folders
A000 0000	Non-resident		\$Index_Allocation	B-Tree structure for larger folders
B000 0000	Either		\$Index_Bitmap	B-Tree structure for larger folders
....				

Table 6: 16-byte Generic Attribute Header structure

Offset (Hex)	Length (Bytes)	Description
0x00	4	Attribute Type Identifier (see table 5)
0x04	4	Attribute Size in Bytes
0x08	1	Non-Resident Flag (0x00 – Resident; 0x01 – Non-Resident)
0x09	1	Length of the attribute stream name (unicode characters)
0x0A	2	Offset to the attribute stream name
0x0C	2	Attribute Flags (see table 7)
0x0E	2	Attribute ID

```
00000000: 4649 4c45 3000 0300 0000 0000 0000 0000 FILE0.....
00000010: 0100 0100 3800 0100 9801 0000 0004 0000 .....8.....
00000020: 0000 0000 0000 0000 0400 0000 4000 0000 .....@....
00000030: 0500 0000 0000 0000 1000 0000 4800 0000 .....H....
00000040: 0000 0000 0000 0000 3000 0000 1800 0000 .....0.....
00000050: 9c3d 5297 57bf ce01 e071 5297 57bf ce01 .=R.W....qR.W...
00000060: e071 5297 57bf ce01 9c3d 5297 57bf ce01 .qR.W....=R.W...
00000070: 2000 0000 0000 0000 0000 0000 0000 0000 .....
00000080: 3000 0000 7000 0000 0000 0000 0000 0300 0...p.....
00000090: 5200 0000 1800 0100 0500 0000 0000 0500 R.....
000000a0: 9c3d 5297 57bf ce01 9c3d 5297 57bf ce01 .=R.W....=R.W...
000000b0: 9c3d 5297 57bf ce01 9c3d 5297 57bf ce01 .=R.W....=R.W...
000000c0: 2000 0000 0000 0000 0000 0000 0000 0000 .....
000000d0: 2000 0000 0000 0000 0800 0000 7400 6500 .....t.e.X.
000000e0: 7400 2e00 7400 7800 7400 0000 1800 0000 t...t.x.t.....
000000f0: 5000 0000 6800 0000 0000 0000 0000 0100 P...h.....
00000100: 5000 0000 1800 0000 0100 0400 1400 0000 P.....
00000110: 2400 0000 0000 0000 3400 0000 0102 0000 $......4.....
00000120: 0000 0005 2000 0000 2002 0000 0102 0000 .....
00000130: 0000 0005 2000 0000 2002 0000 0200 1c00 .....
00000140: 0100 0000 0003 1400 ff01 1f00 0101 0000 .....
00000150: 0000 0001 0000 0000 8000 0000 3800 0000 .....8...
00000160: 0000 0000 0000 0200 1b00 0000 1800 0000 .....
00000170: 5468 6973 2069 7320 6120 736d 616c 6c20 This is a small
00000180: 7465 7874 2066 696c 6521 0a00 0000 0000 text file!.....
00000190: ffff ffff 0000 0000 0000 0000 0000 0000 .....
000001a0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
000001b0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
000001c0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
000001d0: 0000 0000 0000 0000 0000 0000 0000 0000 .....
```

```
## Extracting attribute 3

offset: offset to attribute 2 + size of attribute 2 : 128 + 112 = 240 bytes

dd if=mft-entry-64.dd bs=1 skip=240 status=none | xxd

size: 6800 0000 -> LE to BE -> Hex to Dec -> 104 bytes

dd if=mft-entry-64.dd of=mft-64-attr-3.dd bs=1 skip=240 count=104

## Extracting attribute 4

offset: offset to 3rd attribute + size of 3rd attribute = 240 + 104 = 344 bytes

dd if=mft-entry-64.dd bs=1 skip=344 status=none | xxd

size: 3800 0000 -> LE to BE -> Hex to Dec -> 56 bytes

dd if=mft-entry-64.dd of=mft-64-attr-4.dd bs=1 skip=344 count=56

## Extracting attribute 5

Offset = 344 + 56 = 400 bytes

dd if=mft-entry-64.dd bs=1 skip=400 status=none | xxd

The hexdump indicates that there is no further attributes.
```

Each attribute has a field to specify its size. We use that to find where the attribute ends and the next attribute begins.

An attribute has a

- Generic header (16 bytes) -> specifies the type of attribute
- Extended header -> specifies the offset to the attribute stream and the size of it
- Attribute stream (The data)

The generic header helps us identify what the attribute is and how big it is.

Table 7: Attribute Flags

Hex	Binary	Description
0x00 00	0000 0000 0000 0000	Normal
0x00 01	0000 0000 0000 0001	Compressed
0x40 00	0000 0000 0000 0100	Encrypted
0x80 00	1000 0000 0000 0000	Sparse
0x00 FF	0000 0000 1111 1111	Compression Mark

Table 8: Resident Attribute Extended Header Structure

Offset (Hex)	Length (Bytes)	Description
0x00	16	Generic Attribute Header (see table 6)
0x10	4	Size of the Attribute Stream
0x14	2	Offset to the Attribute Stream

```
### Interpreting Attribute 2
- Type of the attribute: 3000 0000 -> $File_Name
- Is it resident? 00 -> Resident
- Size of the attribute stream: 5200 0000 -> LE to BE -> 00 00 00 52 -> Hex to Dec -> 82 bytes
- Offset to the attribute stream: 1800 -> LE to BE -> 0x18 -> Hex to Dec -> 24 bytes

#### Interpreting attribute stream
- dd if=mft-64-attr-2.dd of=attr-stream.dd bs=1 skip=24 count=82
- Length of the file name: 08 -> Hex to Dec -> 8 Unicode characters
- File name content: 7400 6500 7800 7400 2e00 7400 7800 7400 -> text.txt

-----

### Interpreting Attribute 4
- Type of the attribute: 8000 0000 -> $Data
- Is it resident? 00 -> 00 -> Resident
- Size of the attribute stream: 1b00 0000 -> 00 00 00 1b -> 27 bytes
- Offset to the attribute stream: 1800 -> 00 18 -> 24 bytes

#### Interpreting attribute stream
- dd if=mft-64-attr-4.dd of=attr-stream.dd bs=1 skip=24 count=27
- mv attr-stream.dd text.txt
```

Table 9: Non-Resident Attribute Extended Header Structure

Offset (Hex)	Length (Bytes)	Description
0x00	16	Generic Attribute Header
0x10	8	Starting Virtual Cluster Number of the Run List
0x18	8	Ending Virtual Cluster Number of the Run List
0x20	2	Offset of the Run List
0x22	2	Compression Unit (0x0000 – no compression)
0x24	4	Zero Padding
0x28	8	Physical Size of the Attribute Content
0x30	8	Logical Size of the Attribute Content
0x38	8	Initialised Size of the Attribute Content
0x40	8	Compressed Size of the Attribute Content

```
00000000: 3000 0000 7000 0000 0000 0000 0000 0300 0...p.....
00000010: 5200 0000 1800 0100 0500 0000 0000 0500  R.....
00000020: 9c3d 5297 57bf ce01 9c3d 5297 57bf ce01  .=R.W....=R.W...
00000030: 9c3d 5297 57bf ce01 9c3d 5297 57bf ce01  .=R.W....=R.W...
00000040: 2000 0000 0000 0000 0000 0000 0000 0000  .....
00000050: 2000 0000 0000 0000 0800 7400 6500 7800  ....t.e.x.
00000060: 7400 2e00 7400 7800 7400 0000 1800 0000  t...t.x.t.....
```

Attribute 2

```
00000000: 8000 0000 3800 0000 0000 0000 0000 0200  ....8.....
00000010: 1b00 0000 1800 0000 5468 6973 2069 7320  ....This is
00000020: 6120 736d 616c 6c20 7465 7874 2066 696c  a small text fil
00000030: 6521 0a00 0000 0000  e!.....
```

Attribute 4

Table 10: Standard Information Attribute Stream Structure

Offset (Hex)	Length (Bytes)	Description
0x00	8	Creation Date and Time
0x08	8	Modification Date and Time
0x10	8	\$MFT File Record Modification Date and Time
0x18	8	Last Accessed Date and Time
0x20	4	DOS File Type Flads (see table 11)
0x24	4	Maximum number of versions
0x28	4	Version Number
0x2C	4	Class ID
0x30	4	Owner ID
0x34	4	Security ID
0x38	8	Quota Changed
0x40	8	Update Sequence Number (USN)

Table 11: DOS File Type Flags

Binary	Description
0000 0000 0000 0000 0000 0000 0000 0001	Read-Only
0000 0000 0000 0000 0000 0000 0000 0010	Hidden File
0000 0000 0000 0000 0000 0000 0000 0100	System File
0000 0000 0000 0000 0000 0000 0010 0000	Archive
0000 0000 0000 0000 0000 0000 0100 0000	Volume
0000 0000 0000 0000 0000 0000 1000 0000	Normal
0000 0000 0000 0000 0000 0001 0000 0000	Temporary
0000 0000 0000 0000 0000 0010 0000 0000	Sparse File
0000 0000 0000 0000 0000 0100 0000 0000	Reparse Point
0000 0000 0000 0000 0000 1000 0000 0000	Compressed File
0000 0000 0000 0000 0001 0000 0000 0000	Offline
0000 0000 0000 0000 0010 0000 0000 0000	Content not Indexed
0000 0000 0000 0000 0100 0000 0000 0000	Encrypted

Table 12: File Name Attribute Stream Structure

Offset (Hex)	Length (Bytes)	Description
0x00	6	\$MFT Record Number of the parent directory
0x06	2	Sequence number of the parent directory
0x08	8	Creation date and time
0x10	8	Modification date and time
0x18	8	\$MFT File Record Modification date and time
0x20	8	Last Accessed date and time
0x28	8	Physical Size of non-resident data attribute stream or index allocation stream
0x30	8	Logical size of non-resident data attribute stream or index allocation stream
0x38	4	Dos File Type Flags (see table 11)
0x3C	4	Reparse Value
0x40	1	File Name Length in Unicode characters
0x41	1	File Name Type
0x42	variable	File Name

Table 13: File Name Types

Hex	Binary	Descriptions
0x00	0000 0000	Hard Link or POSIX
0x01	0000 0001	Win 32 Compatible (long file name)
0x02	0000 0010	DOS Compatible (short file name)
0x03	0000 0011	DOS compliant (single file name)

Table 14: Index Root Attribute Stream Structure

Offset (Hex)	Length (Bytes)	Description
0x00	4	Type of indexed attribute (0x3000 0000 for directories)
0x04	4	Collation rule for index sorting
0x08	4	Size of index entry in bytes
0x0C	1	Size of index entry in clusters
0x0D	3	Zero padding (0x000000)
0x10	16	Index Node Header (see Table 15)
*	variable	

Table 15: Index Node Header Structure

Offset (Hex)	Length (Bytes)	Description
0x00	4	Offset to the first index node entry
0x04	4	Logical size of the index node entry stream
0x08	4	Physical size of the index node entry stream
0x0C	1	Child node flag (0x01 Index Allocation Attribute needed)
0x0D	3	Zero Padding (0x000000)

Table 16: Volume Information Attribute Stream Structure

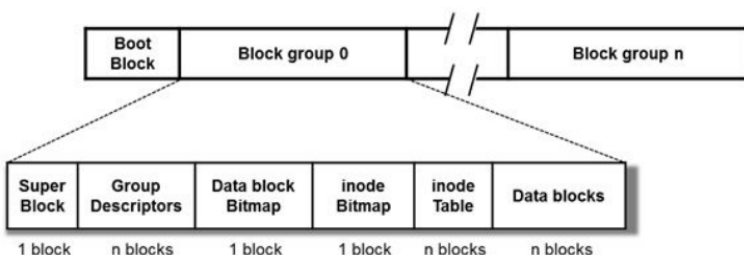
Offset (Hex)	Length (Bytes)	Description
0x00	8	Zero Padding
0x08	1	Major NTFS version number
0x09	1	Minor NTFS version number
0x0A	2	Volume flags
0x0C	4	Zero Padding

Table 17: Volume Flags

Hex	Binary	Description
0x0001	0000 0000 0000 0001	Dirty Flag
0x0002	0000 0000 0000 0010	Resize Log File
0x0004	0000 0000 0000 0100	Upgrade on Mount
0x0008	0000 0000 0000 1000	Mounted on NT 4
0x0010	0000 0000 0001 0000	Delete USN Underway
0x0020	0000 0000 0010 0000	Repair Object IDs
0x8000	1000 0000 0000 0000	Modified by <i>chkdsk</i> .

07. EXT File System

- Relatively simpler and highly reliable file system.
- The extended FS is the default FS used in Linux
- Versions -> ext, ext2, ext3, ext4
- Ext3 and Ext4 are journaling file systems (keeps track of pending changes)
- Latest versions of GNU/Linux distributions have ext4.
- Manual inspection is possible using the data structures of each component.
- Blocks: a collection of sectors
 - Each block has an address starting with 0
 - Blocks are grouped together, and the groups are also addressed from 0
 - The allocation status of blocks are given in the data block bitmap

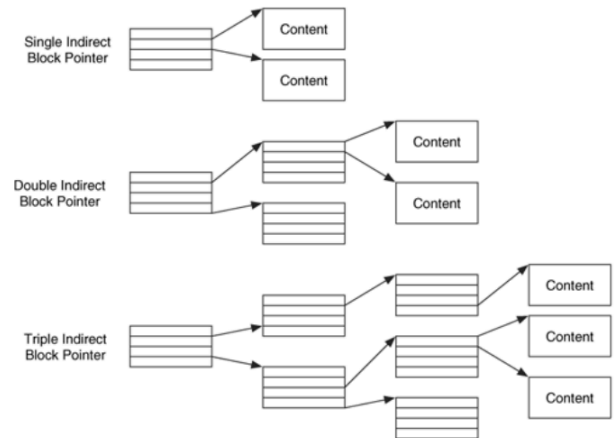


- Boot block: provides basic info about partition
 - ext2/ext3 -> 1024 bytes
 - ext4 -> 4096 bytes
 - Not useful for manual inspection
- Block groups: separate collections of blocks.

- Super block: contains the info of the whole file system (size, total inodes...)
 - Replicated in every block group
 - Very first superblock is right after the boot block
 - Contains the block size, total num of blocks, num of blocks per block group, num of reserved blocks before the 1st block group.
 - Contains the total num of inodes, and the num of inodes per block group.
 - Defines what features the FS has enabled.
 - For ext2,ext3: superblock is located at byte offset 1024, with 1024 bytes.
 - For ext4, superblock is located at byte offset 4096, with 1024 bytes.
- Block group descriptors: a table containing an entry for each block group.
 - Each entry is 32 bytes long.
 - Contains separate entries to describe useful information about each block group.
 - Located after the superblock in a block group.
 - It is included as a redundant copy in each block group.
- Data block bitmap: file and directory content is stored in blocks, and the allocation of each block is stored in a block bitmap. Every block group has a bitmap for its blocks.
- inode bitmap: the allocation status of inodes.
- inode table: an array of inode data structures.
 - An inode will contain the critical details about a file or directory stored within that block group.
- Data blocks: the actual blocks used to store data of files and directories.
 - The blocks are allocated to inodes.

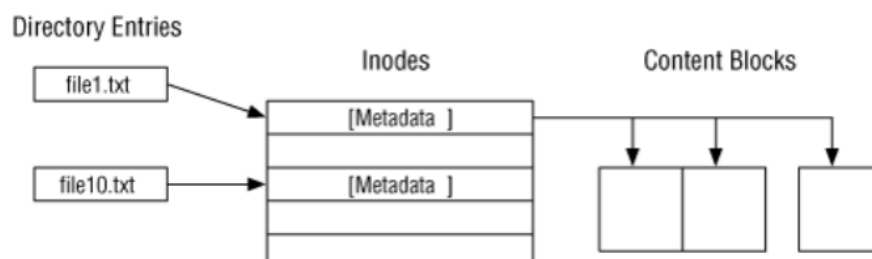
inodes

- The metadata record for a file or directory.
- Each inode stores the file size, owner and permissions, timestamps (created, modified, accessed), pointers to data blocks where the actual content is.
- Each inode has an address starting with 1.
- All inodes are the same size.
- A set of inodes are allocated to each block group.
- inode number 2 -> reserved for the root directory.
- The allocation status of inodes are given in the inode bitmap.
- Each inode has a set of pointers to the allocated data blocks to it to contain file or directory data.
- Some data block pointers are direct, and some are indirect at multiple layers.
- inodes find its data with a layered pointer system:
 - Direct pointers
 - Single indirect pointers
 - Double indirect pointers
 - Tripe indirect pointers



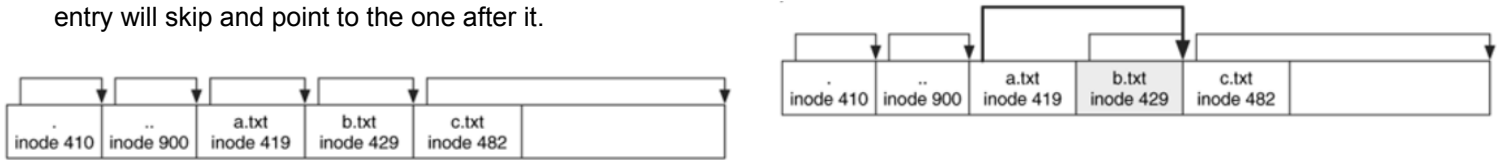
Files and Directories in EXT

- An inode in inode table represents a file or directory.
- If an inode represents a directory, then the data blocks contain directory entries.
 - Each directory entry contains a file or directory name under the current directory and points to the corresponding inode number
- If an inode represents a file, its data blocks have the file content.
- So the name of the file or directory is only stored in the parent directory's inode's data blocks' directory entry.



Directory Entries

- Each entry → file or directory name + its inode number
- Every directory starts with . (current) and .. (parent) directory entries
- Entries have a dynamic length (because the file name can be from 1 to 255 characters long), so the data structure has a field that identifies how long the name is and where the next directory entry is (pointer to the next dir entry)
- When a particular directory entry is deleted, the previous entry will skip and point to the one after it.

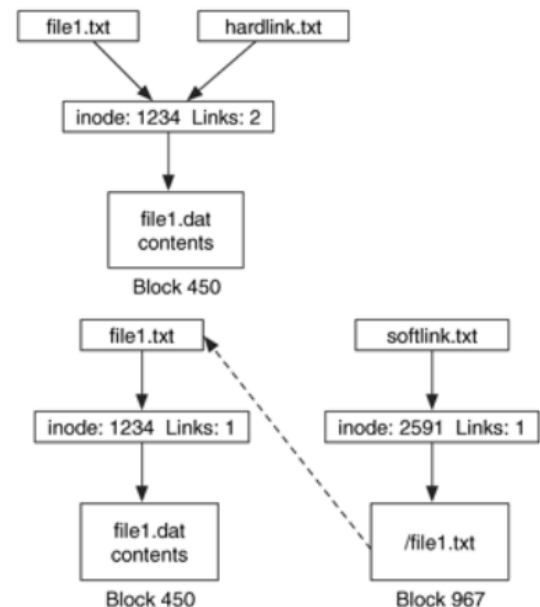


- When a directory entry is added, compare file/directory name lengths against the pointer to the next entry to see if there are unused (skipped) space.

Hard Link

- Just creating a second name that points to the exact same inode

```
/home/documents/  
├─ essay.txt → inode 204 ← same inode!  
├─ hardlink.txt → inode 204 ← same inode!  
  
inode 204 (Links: 2) → data blocks ("Once upon a time...")
```



Soft Link

- A completely separate inode whose content is just a path pointing to the original file.

```
/home/documents/  
├─ essay.txt → inode 204  
├─ softlink.txt → inode 591 ← DIFFERENT inode  
  
inode 204 (Links: 1) → data blocks ("Once upon a time...")  
inode 591 (Links: 1) → data block contains "/home/documents/essay.txt"  
↑ just a text path, nothing more
```

- ln → create hard and soft (symbolic) links to files
- ls -li → view inode numbers

```
# Create a hard link  
ln essay.txt hardlink.txt  
  
# Create a soft link  
ln -s essay.txt softlink.txt  
  
# View inode numbers with ls -li  
ls -li  
  
# Output:  
204 essay.txt  
204 hardlink.txt ← same inode number as essay.txt (hard link)  
591 softlink.txt ← different inode number (soft link)
```

Mounting File Systems

- A filesystem can be mounted inside a directory of another FS.
- Files originally under that directory become invisible while it's used as a mount pedal.

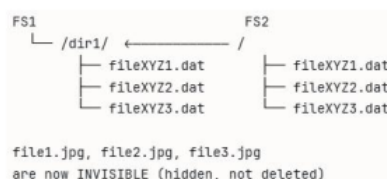
mount → inspect mounted filesystems

```
# Mount a USB drive to /mnt/usb/  
mount /dev/sdb1 /mnt/usb/  
  
# Mount a disk image (common in forensics!)  
mount disk.img /mnt/evidence/  
  
# See all currently mounted filesystems  
mount
```

Before mounting:



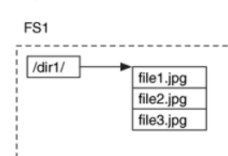
When mounting FS2 onto /dir1/:



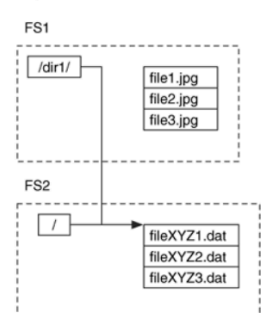
After unmounting FS2:



A)



B)



08. Autopsy and Sleuthkit for FS Forensics

- Manual inspection is not easy.
- So forensic tools can be used, like X-Ways Forensics, AccessData FTK, EnCase etc.
- But their software license is expensive, so we can ideally use free and open source software. (Sleuth Kit and Autopsy).

Autopsy

- The GUI counterpart of Autopsy. Basically a wrapper around the Sleuth Kit.
- It is a complete software providing facility to manage disk images, extracting files, analysing disk images and files, and more.
- Highly extensible with various official and third party modules for things like timeline analysis, keyword search, data carving, multimedia analysis etc.
- The latest version is always available for Windows. (Linux and Mac OS X supported versions are old)

Sleuth Kit

- A collection of command line tools used to analyse disk images and file systems.
- The best one in the FOSS world.
- Can even write shell scripts wrapping sleuthkit commands to automate FS analysis routines.
- **mmls** → used to inspect a disk image (**mmls /path/to/disk-image**)
 - Shows whether the disk is partitioned with DOS partitioning (makes MBR partition table) or UEFI partitioning (makes GUID Partition Table – GPT), the starting and ending positions of each partition, and the FS type of each partition
- **fsstat** → used to inspect a partition
 - **fsstat -f list**, **fsstat /path/to/partition-image**, **fsstat -o offset-to-partition /path/to/disk/image**
 - Reads the info from the boot sector (FAT), \$boot file (NTFS), or superblock (EXT) of the FS.
 - Can automatically detect the FS most of the time, and if not, we can manually specify the FS.
- **fls** → used to list files and directories
 - **fls /path/to/partition-image**, **fls -p -r -o offset-to-partition /path/to/disk-image**, **fls -p -r -d -o offset-to-partition /path/to/disk-image**
 - Detects the file systems and list the files and directories including deleted (but surviving) files.
 - Output: whether it's a file or directory, deleted or not, and the unique entry/inode number.
 - entry/inode number is useful to extract or recover the file later.
- **istat** → view details of a file or directory (the inode number for the command can be taken from fls command)
 - **istat /path/to/partition-image inode-number**, **istat -o offset-to-partition /path/to/disk-image inode-number**
 - Detects the FS and identifies the metadata of a file/dir specified by its entry/inode number.
 - Output: file type, name, size, timestamps, and sectors allocated to the file contents.
- **icat** → extracting/recovering a file or directory
 - **icat /path/to/partition-image inode-number > file-name**, **icat -o offset-to-partition /path/to/disk-image inode-number > file-name**
 - Detects the FS and copies the file contents to the standard input.
 - We can redirect the standard output so that the file contents can be saved into a new file we make.
 - This is not very useful for a directory.

```
dlnu@dlnu-Aspire-A515-56G:~/Documents/Y4S2/DF/DF_Assignment$ mmls file-system-as-
signment-image.dd
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

    Slot      Start      End      Length    Description
000:  Meta      0000000000  0000000000  0000000001  Primary Table (#0)
001:  -----  0000000000  0000002047  0000002048  Unallocated
002:  000:000  0000002048  0000053247  0000051200  Linux (0x83)
003:  000:001  0000053248  0000102399  0000049152  Linux (0x83)
```

The TSK 4 command list

- **blkcalc** - Converts between unallocated disk unit numbers and regular disk unit numbers.
- **blkcat** - Display the contents of file system data unit in a disk image.
- **blkls** - List or output file system data units.
- **blkstat** - Display details of a file system data unit (i.e. block or sector).
- **fcats** - Output the contents of a file based on its name.
- **ffind** - Finds the name of the file or directory using a given inode.
- **fiwalk** - print the filesystem statistics and exit.
- **fls** - List file and directory names in a disk image.
- **fsstat** - Display general details of a file system.
- **hfind** - Lookup a hash value in a hash database.
- **icat** - Output the contents of a file based on its inode number.
- **ifind** - Find the meta-data structure that has allocated a given disk unit or file name.
- **ils** - List inode information.
- **img_cat** - Output contents of an image file.
- **img_stat** - Display details of an image file.
- **istat** - Display details of a meta-data structure (i.e. inode).
- **jcat** - Show the contents of a block in the file system journal.
- **jls** - List the contents of a file system journal.
- **jpeg_extract** - jpeg extractor.
- **mactime** - Create an ASCII time line of file activity.
- **mmcat** - Output the contents of a partition to stdout.
- **mmls** - Display the partition layout of a volume system (partition tables).
- **mmstat** - Display details about the volume system (partition tables).
- **sigfind** - Find a binary signature in a file.
- **sorter** - Sort files in an image into categories based on file type.
- **srch_strings** - Display printable strings in files.
- **tsk_comparedir** - compare the contents of a directory with the contents of an image or local device.
- **tsk_gettimes** - Collect MAC times from a disk image into a body file.
- **tsk_loaddb** - populate a SQLite database with metadata from a disk image.
- **tsk_recover** - Export files from an image into a local directory.

09. Mobile Forensics

- Crimes involving mobile devices:
 - Mobile phone theft
 - Plotting, organising and coordinating a crime
 - Data theft - using a phone to take photos of confidential info and strong stolen data
 - Identity theft - using a stolen phone to represent the identity of the owner
 - Harassment - threatening emails, SMS, images
- Mobile forensics → a branch of DF relating to recovery of digital evidences or data from a mobile device under forensically sound conditions

Computer Forensics	Mobile Forensics
Can be powered off when not in use	Almost always stays powered on
Basically a processor & volatile/non-volatile storage, and network interface	Much more than a computer. Contains lots of sensors and built-in GPS
Limited number of OS	Limited number of OS
Standard hardware & peripheral configurations	Manufacturer specific hardware & peripheral configs
Easy to access the non-volatile storage for forensic imaging	Difficult to access the non-volatile storage for forensic imaging

Acquisition Types:

- Manual: Looking through the UI (screen by screen).
- Logical: Extracting files and folders using APIs/Sync protocols.
- Physical: A bit-by-bit copy of the flash memory. This allows for recovery of deleted data but is difficult due to hardware locks.
- Key sources of evidence (where the data related to a mobile device is stored):
 - Cellular Network Service Provider, SIM card, SD card, Internal (flash) storage, Cloud storage

Cellular Network Service Provider

- Has logs at various stages of their infrastructure related to a mobile device and its subscriber.
- Can identify physical loc & movements based on the cell towers that the device has connected to.
- Other mobile device users who have been contacted (calls and texts) can be traced.
- Needs a court order to access these.

SIM (Subscriber Identity Module) Card

- A plastic card that embeds a tiny microprocessor, mem, & physical contact points for I/O.
- Given by cel net serv prov & can be placed inside a mobile phone to use it with the specific service provider.
- IMSI** → International Mobile Subscriber Identity. Unique num stored in SIM that identifies the user in the mobile network.
- ICCID** → IC Card ID. A serial num printed on the SIM card.
- IMEI** → International Mobile Equipment Identity. Unique num assigned (hard-coded) to mobile phone.
- There are 4 available sizes → Full size (Credit card size), Mini SIM (standard size), Micro SIM, Nano SIM.
- The physical dimension can cause compatibility issues with the card reader. But contact points on cards are identical.
- EEPROM memory has a small FS:
 - Master File (MF) → Root of the FS
 - Dedicated Files (DF) → Directories
 - Elementary Files (EF) → Data files
- Some EF → EF ADN (abbreviated dialing num), EF FPLMN (Forbidden Public Land Mobile Network), EF LND (Last Number Dialed), EF LOCI (Last Connected Location), EF SMS (send and received text msgs)
- Details in SIM can be read using a mobile device (with proper software) or a dedicated SIM card reader.
- Mobile forensic software: MOBILedit (can read SIM data through SIM card reader hardware)
- There are SIM card cloning hardware/software as well.
- Cloning in this case means copying IMSI and the Authentication Key to a new SIM card.

SD (Secure Digital) Cards

- A standard non-volatile, highly portable storage device that uses FAT/exFAT FS. so it's good for FS forensics.
- Can find Application and system-related data of mobile devices in these
- Easy to use (but also easy for malicious users to discard them)

Internal (flash) Storage

- A flash chip built-in to the circuit board, commonly referred to as internal memory.
- In modern devices, this flash chip uses Embedded MultiMediaCard (eMMC) technology mostly.
- The OS, pre-installed apps, and new apps go into this.
- Usually, eMMC is attached permanently to the circuit board of a phone. So the safest option is to do a logical extraction (copying stored files instead of taking a forensic image).
- After connecting mobile to PC, can access the device using Android Debug Bridge (ADB) interface to copy.
- Rooting the device may be needed (breaking storage protection)
- If taking a forensic image, chip-off forensics (taking eMMC out and placing it in a special circuit board to read content) can be done or wiretapping (without detaching eMMC) and reading using JTAG interface.
- These two methods are risky and may violate ACPO guidelines.

Cloud Storage

- Apps store data in cloud servers. OS and apps backup data to cloud storage.
- Can identify user's online accounts and perform cloud forensics to recover dig evidence of smartphone.
- Procedure of handling a mobile device:
 - Posses a warrant from the applicable legal authority to search and seize devices.
 - Accompany a digital forensic examiner when executing the warrant.
 - Take photos of the physical condition of the device.
 - If the device is powered off, leave it off. If powered on, leave it on. To avoid encryption locks or losing RAM data. If a turned on device is turned off, may trigger a "Secure Startup" or encryption lock, then impossible to access without a passcode. Ensure it stays powered on by connecting it to a portable power bank.
 - If powered on, place inside a faraday bag. (blocks EM signals. Prevents a remote wipe command) / airplane mode.
 - Stick with ACPO guidelines (have acceptable reasons if deviating).

10. Tools for Mobile Forensics

- If it's an SD card, normal FS forensic tools can be used.
- To get smartphone specific data from its internal storage,
 - MOBILedit, Oxygen Forensic Suit, Elcomsoft iOS Forensic Toolkit, Passware.
- MOBILedit: proprietary closed-source software tool.
- Recoverable data from a smartphone:
 - Address books, calendars, call logs, emails, images/graphics, IMSI, IMEI, SMS, user files, videos...

Challenges and Solutions

- Can have destroyed devices – can use chip-off forensics
- Remote wiping – can use Faraday bag/cage
- Device diversity with different hardware & software configs – continuous staff training on new devices
- Locked/encrypted devices – Ask for password/key through legal procedure. If that fails, use EMSCA.
- Hard to stay up to date with the changing mobile device market.

Camera Ballistics

- If you use the camera within an app, the metadata and quality might differ from the native camera app.
- Social media apps like WhatsApp usually strip out original metadata and compress the image to save space. This makes "proving" the origin much harder compared to an original file from the phone's DCIM folder.
- EXIF Data (Exchangeable Image File Format) → a set of metadata embedded in the image file.
 - Tells the cam make/model, shutter speed, focal length, GPS coordinates (if enabled) and the date/time the photo was taken.
 - EXIF is incredibly easy to forge, strip, or alter using simple "EXIF editor" tools. So cannot rely.
- Photo Response Non-Uniformity (PRNU)
 - No two digital camera sensors are identical. Due to microscopic imperfections during the manufacturing of the silicon (the CCD or CMOS sensor), some pixels are slightly more sensitive to light than others.
 - This creates a consistent, unique pattern of noise that is overlaid on every single image that specific sensor captures. It is invisible to the naked eye but can be extracted mathematically.
 - Unlike EXIF data, PRNU noise cannot be easily removed or forged without destroying the image quality. So the investigator can say, "This specific photo was taken by *this exact* iPhone 15, and not another identical model."

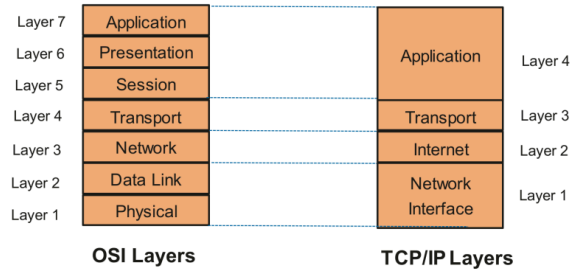
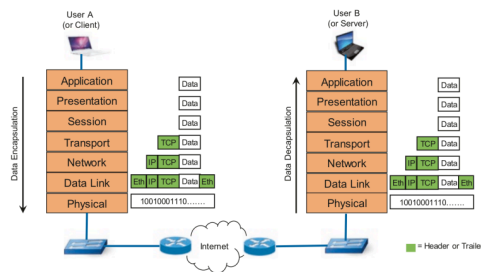
IMSI-Catchers

- Hardware and software tools used to closely monitor a mobile device while it is in use (not after seizing).
- 4G LTE GSM networks use data encryption when comm between mobile device and cell tower.
- A rogue base station is used as a MITM attack. Mobile phone is tricked to send unencrypted data through it.

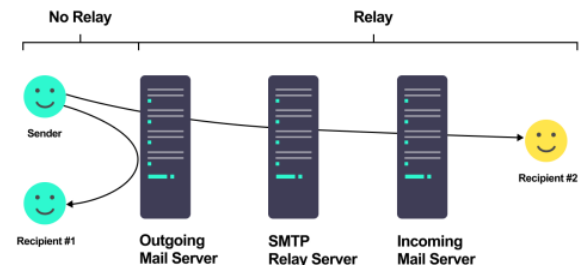
11. Introduction to Network Forensics

- Acquisition of forensic evidence from network-related data sources in computer systems to assist in legal and cooperative investigations.

Open Systems Interconnections (OSI) Model



- IP → network layer. Connects hosts across multiple hops. IPv4 (32-bit addr), IPv6 (128-bit addr)
- TCP (Transmission Control Protocol) → Multiplexes comm to dif processes in host. Connection-oriented. Reliable. Sequencing of segments. Port number is 16-bits.
- UDP (User Datagram Prot) → Multiplexes comm to dif processes in host. Connectionless. Unreliable. Segments can be delivered out of order. Port number is 16-bits.
- HTTP → TCP port 80 or 443. Client server arch. Serving hypertext documents. HTML content can be processed both at server and client side with scripting languages.
- SMTP → sends emails b/w end-user devices. The intermediate email forwarders are Mail Transfer Agent (MTA) servers (TCP port 25). Email transfer b/w MTAs: done by SMTP
 - Sender and recipient uses IMAP or POP3 to get connected to their own MTA.
 - End-to-end encryption is possible but out of scope of SMTP.



- Sources of Data:
 - On the wire: network traffic going through cable
 - In the air: going through wireless medium like WiFi
 - Switches: have Content Addressable Mem (CAM) tables. Maps phys ports to net interfaces in LAN.
 - Routers: have routing tables. Maps ports to net interfaces of multiple LANs.
 - DHCP server: keep logs on IP addresses assigned to MAC addresses for a lease time duration.
 - Name servers: keep logs on resolving IP addresses to dom names and who requested the resolution.
 - Auth. servers: logs the successful and failed login attempts.
 - Intrusion Prevention Systems & IDS: logs details of intrusive actions to the internal network.
 - Firewalls: controls/prevents dif types of traffic into internal net. Some have a logging mechanism.
 - Web proxies: stores lots of info (websites accessed by different hosts as well)
 - Application servers: stores application-specific data.
 - Central log servers: logs a wide variety of events from the whole network.
 - Websites: directly browsing them to get valuable info.

Evidence Types

- Full Packet Capture (Pcap): Every bit of traffic (e.g., using Wireshark or tcpdump).
- Flow Data (NetFlow): Metadata about connections (Source/Dest IP, ports, duration) without the actual content.
- Log Files: Web server logs (Apache/Nginx), Firewall logs, and Proxy logs.

Challenges in Network Forensics

- FS forensics have limited FS. but network forensics have lots of protocols. Hard for one person to learn all.
- FS forensics is done on a limited num of storage devices. In this, there are lots of dif hardware devices that have proprietary components, which are very different from each other.
- Too many potential sources of evidence. Hard to locate the right evidence source at the right time.
- Partly live data (traffic analysis), and partly offline data (log file analysis)
- Cannot do forensic imaging for live data.
- Network devices don't store complete info. They log very coarsely granular info.
- Network devices have limited non-volatile storage. So most info is kept in volatile form.
- Seizing a network device under investigation is too complicated and impractical.
- FS based evidence is well recognised in legal proceedings. Network base evidence is not (yet).

Scenario 1:

The internal network of a company is significantly slowed down, despite the fast bandwidth they are supposed to have. A potential botnet malware infection is suspected.

- Search for suspicious executable files in all the important devices in the network (i.e., file system forensics).
- Check the memory (RAM) of devices and computers connected to the network for potential memory-based malware (i.e., memory forensics).
- Capture live traffic at important locations of the internal network and look for hints of unauthorised devices/processes communicating through the network (i.e., network forensics).
- Inspect log files of network devices and servers for any hits of unauthorised devices/processes operating in the network (i.e., network forensics).

Scenario 2:

Some important files in a server open to the Internet have been deleted or modified. We don't know how it happened and who did it.

- Take a forensic image of the server's disk and look for timestamps and other hints of which user may have accessed those files (i.e., file system forensics).
- Check the memory (RAM) of the server for potential hints of which user, when, and how the files were deleted (i.e., memory forensics).
- Inspect log files of network devices and server for any hits of someone remotely accessing those files (i.e., network forensics).

Scenario 3:

We have a suspect in a financial fraud investigation whose laptop has been seized by the law-enforcement. We need to know his/her activities, such as online communications this person did, specific online services/website this person has used, etc.

- Take a forensic image of the laptop's disk and look for relevant files such as web browser's and email client's logs and temporary files (i.e., file system forensics).
- Check the memory (RAM) of the laptop for potential hints of passwords this person may have entered to log into various online services, such as email accounts, websites, etc. (i.e., memory forensics).
- Inspect the temporary files and logs with specific network application related data according to their specifications for online activities and later logging into those services to acquire data directly (i.e., network forensics with the help of information found in file system and memory forensics).
- Directly contacting online service providers through the legal system to acquire online activity data and analysing them (i.e., network forensics).

Scenario 4:

During a file system analysis of a suspect, we found a document that mentions an email address of a collaborator. We suspect that the owner of this email address is also related to the crime under investigation. So, we need to find the owner of this email address.

- Perform a keyword search on the forensic image of the suspect's laptops for other mentions of this email address (i.e., file system forensics).
- Analyse the email account of the suspect for potential emails sent and received to the aforementioned email address from the original suspect (i.e., network forensics).
- Directly contact the email service provider through the legal system to get the details of the owner of the email address (i.e., network forensics).
- Perform an open-source intelligence (OSINT) search for the appearance of this suspicious email address in various other places on the internet, such as websites, public forums, social media, etc. (i.e., network forensics).

12. Network Layer Evidence for Network Forensics

- Evidence Acquisition in network forensics can be passive or active.
- In passive: network traffic is intercepted in the physical medium and saved to the investigator's storage.
 - Requires tampering into the physical medium (physical interception)
- In active: investigator directly interacts with network devices to get evidence.

Physical Interception:

- Intercepting traffic in cables: inline network taps, vampire taps, induction coils, fibre optic taps.
- Intercepting hubs: broadcasts to all ports. Connecting to 1 available port gives access to all traffic. The investigator device should be in promiscuous mode.
- Intercepting switches: forwards to only relevant ports. Connecting to 1 port doesn't receive other traffic. Configuring for port mirroring through admin access. Attacking with MAC flooding and ARP spoofing.
- Intercepting wireless traffic: investigator's devices must be in monitor mode. SSIDs, MAC addr, etc. can be visible. Traffic is captured first and encryption is broken later.

Passive Network Data Acquisition Software

- Libpcap → UNIX C library. Gives an API to capture and filter data link layer frames.
- Winpcap → library based on Libpcap for Windows.
- tcpdump, Wireshark, tshark, Snort → packet sniffing and analysing.
- nmap → port scanning
- ngrep → searching in packet traces
- mergecap → merging packet captures
- tcpflow → split a capture by TCP stream
- netstat → info on open ports of a host
- traceroute → trace the path to a host
- nslookup → querying name servers

Berkeley Packet Filter (BPF) Language

- Included in Libpcap to filter and select specific traffic patterns. Almost all libpcap based utilities support this.
- Primitives (can make simple filters):
 - type → what kind of thing the id/number refers to (host, net, port, portrange)
 - dir → packet transfer direction (src, dst)
 - proto → restrict the protocol (ether, fddi, tr, wlan, ip, ip6, arp, rarp, tcp, udp)
- Have logical and, or, not operations.
- Ex: **src host 192.168.1** → packets originating from this ip are selected.
- Ex: **udp dst port not 53** → UDP packets that aren't destined to this port number are selected.
- Packet sniffers can usually capture, decode, and visualize packet content. But some malicious attacks may use custom made protocols on top of IP protocol. Packet sniffers and network analyser tools may fail to parse them. Those packets have to be manually inspected using a hex viewer.
- Active acquisition: log files are stored in net devices and can be used as evidence. Networking devices can be accessed via various interfaces to reach these log files.
 - Ex: console login, secure shell (SSH), secure copy (SCP), SSH file transfer Protocol (SFTP), Telnet, Simple Network Management Protocol (SNMP), web and proprietary interfaces.
- Forensic Readiness in Network Forensics:
 - Packet sniffing → only useful when applied during an ongoing incident.
 - Ideally, an org's network infrastructure should be forensic ready with tools configured to capture traffic
 - There are commercial network forensic tools that continuously capture net traffic.
- Key network layer evidence:
 - IP Addresses: Identify source and destination.
 - TTL (Time to Live): Can help identify the OS of the sender or indicate the number of hops (distance) to a suspect.
 - Can detect spoofing and anomalies: If an investigator sees packets supposedly coming from the same internal IP address but with wildly different TTL values (e.g., one is 64 and another is 128), it is a strong indicator of IP Spoofing or that multiple different devices are behind a single gateway.
 - Can map network topology: By analyzing TTL values from various points in the network, forensics experts can reconstruct the path an attacker took, identifying which routers or firewalls the malicious traffic successfully bypassed.

13. Application Layer Evidence for Network Forensics

Criminal uses of internet services

- Can impersonate through email, web, social media.
- Sexual offences (through chat networks)
- Stalking
- Data theft (breaking into private networks to steal credit card numbers and trade secrets)
- Hate groups (communicate, publish, and threaten)
- Things to be careful about → proxies, bots, encryption, anonymous and pseudonymous emails, Tor network, Invisible Internet Project (I2P), VPNs, Cryptocurrency

Email Forgery and Tracking

- Email forgery is ineffective because they usually contain the sender's IP address.
- The local MTA puts the current time, MTA name, and some technical info at the top of the email message. This is the "Received" header. Then that message is passed from 1 MTA to another until the destination.
- Email forgery takes advantage of how MTAs exchange messages using SMTP. Using 4 fields in SMTP (helo, mail from, rcpt to, data), 1 MTA can instruct another MTA to pass an email message to the destination.
- Using the same fields, someone can command an MTA directly using a Telnet utility program by typing "telnet mta.sending.com 25". This tells Telnet to connect to port 25 on MTA and permit SMTP commands to be types and sent to the MTA. (netcat can be used instead of telnet as well)
- The alert examiner will see that the forged Received header is inconsistent with other headers. An intermediate header may reveal the sender's actual IP address where they connected through telnet.
- The ISP responsible for the sender's IP address can then check which user account was used for this.
- So because of this, email forgers send messages to an SMTP relay via a proxy. Then it is difficult to know the originating IP address because the web proxy conceals this information.
- To track emails, the 2 most useful email headers are "Message-ID" and "Received" headers.
- Message-ID → globally unique. Cannot be always relied on. Can be forged.
- Received headers → can insert forged Received headers to a message but some of the headers at the top of the message must be valid because they were added by MTAs that delivered the message.
 - Sometimes, it would contain the sender's email address or IP address of the originating computer.

Key Application Layer Evidence

- Web Logs: Identify specific URLs visited, user-agents, and HTTP status codes.
 - Identifying Attacker: Logs record the Source IP Address. allows investigators to trace request back to specific network or geographic location.
 - Determining Intent: By looking at the Request URI (the page or file requested), investigators can see if an attacker was trying to access sensitive areas like /admin or /config.php.
 - Analyzing Success/Failure: The HTTP Status Code reveals what happened.
 - Tool Identification: User-Agent string can reveal browser or automated tool (sqlmap or nmap) used by the suspect.
- Email Forensics: Analyze SMTP headers to find the original sender's IP address and relay servers.

14. Open Source Intelligence for Digital Forensics

- OSINT (Open Source Intelligence) – process of gathering info from publicly available sources on internet.
 - Easiest most straightforward way to gather info. And no legal authority is needed to do an OSINT investigation.
 - Conducting OSINT takes time. Better if the process can be automated.
 - Sources: Social media, public records, WHOIS data (for domain ownership), and the Dark Web.
 - Use in Investigations: Identifying a suspect's online aliases, tracking their movements via public photos, or finding leaked credentials.
 - OSINT tools → automates the process and reduces manual work. → Spiderfoot, Recon-ng, Shodan
- Spiderfoot → open source OSINT automation tool.
 - Has an embedded web server to provide a clean and intuitive web based interface.
 - Can also be used via command line.
 - Has over 200 modules to search various info on the internet. Can even search on the dark web through the Tor network.
 - Can find: IP address, dom/sub-dom name, hostname, network subnet (CIDR), email address, phone number, username, person's name, bitcoin address.
- Recon-ng → free and open source reconnaissance/OSINT tool.

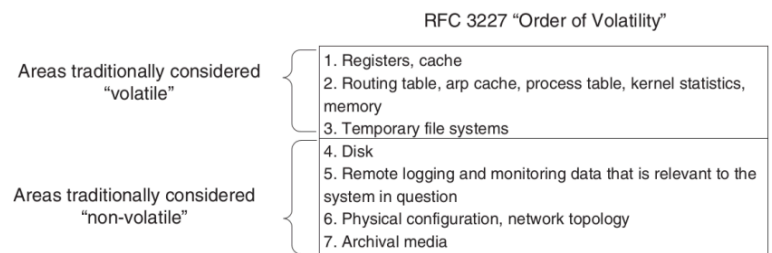
- Can configure, perform recon, and output results to different report types.
 - Has dif modules with dif capabilities. Can decide which modules to unleash.
- Shodan → search engines for Internet-connected devices.
 - Can search for servers, DBs, routers, CCTV cameras, webcams, IoT devices...
 - Helps identify open ports, OSs, and other system info of such devices.
 - Info is collected by sending requests to devices and analysing headers in the responses.

OSINT sources an investigator should check to find the owner of a malicious website used for phishing.

- WHOIS Lookup: To find the domain registration details, such as the registrar, name of the owner (if not hidden), and email addresses.
- Wayback Machine (Internet Archive): To view historical versions of the website to see if the owner accidentally left personal details or different contact info in the past.

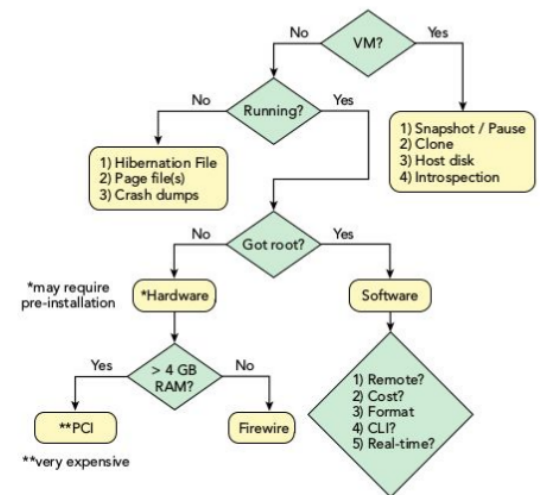
15. Introduction to Memory Forensics

- Lots of OS specific data are not permanently saved in storage. pwds and keys are not stored in plain text.
- There are malware that doesn't install but stays in memory.
- Stack → temp data associated with executing functions (local variables, function return addresses)
- Heap → stores the process's data that needs to be dynamically allocated.
- RFC3227 lists potential data sources with the order of volatility.
- Certain commands and utilities can be used to extract specific info from the volatile memory of a compromised system. But these might not provide a lot of info.. And if some info was forgotten to be extracted, it can never be extracted later.
- The ideal approach → Dumping the entire RAM (can be analysed later with sophisticated tools)
 - Needs the support of hardware or the OS of the target system.
 - Often conducted through the help of kernel modules or special toolkits.
- Memory Forensics → Extraction and analysis of volatile memory of computer systems.
 - Live operation (like network forensics)
 - Time sensitive task (like network forensics)
 - Slightly intrusive task and involves a lot of risk since the system can crash.



16. Volatility for Memory Analysis

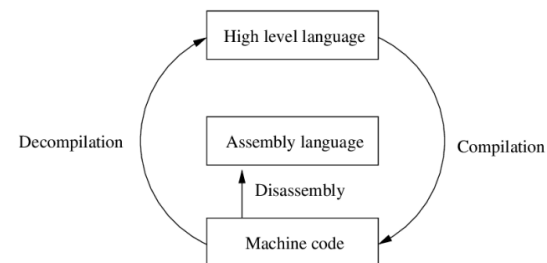
- Memory acquisition: copying volatile memory contents to non-volatile storage for preservation.
 - 2 procedures: H/w based and s/w based.
- Hardware based:
 - Access mem without relying on OS. suspend the CPU and with Direct Memory Access (DMA) to copy contents of the physical memory.
 - FIREWIRE/IEEE allows client devices for direct access to host mem, bypassing the OS.
- Software based:
 - Trusted toolkit has to be used to collect volatile data.
- Procedure depends on the exact OS that's running.
- Tools: WinPmem (Windows), Belkasoft RAM Capturer (Windows), AVML (Linux), LiME (Linux)
- Linux kernel has 2 virtual devices: entire phys mem (/dev/mem), entire virtual memory (/dev/kmem)
- When the Kernel Lockdown feature is enabled. It severely restricts the access to those special files.
- LiME (Linux Memory Extractor) → loadable kernel module that expands Linux's kernels capacity into mem extraction.
 - Kernel module: an extra kernel (not a part of monolithic kernel). Can load and unloaded during run time.
 - After memory img is extracted,
 - Extract all strings in mem dump, search for specific keywords in a map.



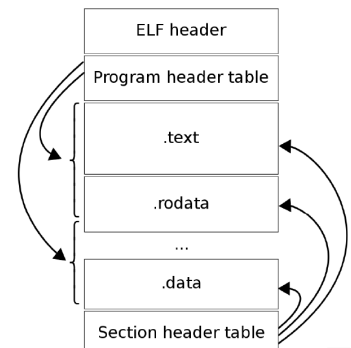
- Volatility: a framework to extract forensic artefacts from memory images.
 - A single tool to cover Windows, Mac, Linux, and android memory images.
 - A memory image can be dissected using this.

17. Malware Forensics

- Types of malware: Ransomware, Spyware, Adware, Trojan, worms, virus, rootkits, keyloggers, bots/botnets
- Malware forensics purposes:
 - To understand the malware to develop antivirus/IDS signature.
 - To detect compromised systems on the network.
 - To contain an infection.
 - To track down the source of the malware.
 - Gathering evidence for courts
- Tactics used by malware: encryption and obfuscation, distributed and control systems, automatic self update. Irregular network behavior
- Investigating on malware can be performed on multiple fronts → Observing net behavior (partly net forensics), inspecting executable code (partly FS forensics), Inspecting memory behavior (partly mem forensics)
- For a network based malware, a measurable difference may exist in the network traffic between the before and after of an infection.
- Approaches → Static Analysis: Examine malware without running it (checking strings, headers, reverse engineering). Dynamic Analysis: Running malware in a controlled "sandbox" to observe its behavior (what files it creates, what IPs it contacts).
- 3 aspects to look for of the network behavior of malware → Propagation, C&C communication, payload.
- Propagation: malware can spread into the new systems through various means. (email, web links and content, direct network based exploitation.)
 - Sniffing net traffic & looking for signatures of malware files in TCP streams can be used to catch the speed/propagation of malware.
- C&C Communications:
 - C&C channels allow remote attacks to manage and update infected systems.
 - HTTP, Social Networking Sites, Peer-to-peer, Channels using Internet Relay Chat (IRC) protocol
 - Noticing statistical difference in the network traffic can hint that a malware is
 - maintaining a C&C communication channel, Volume, Directionality (source and destination), Timing of flow patterns.
- Payload: post infection network behavior of a malware depended on its designer's intention.
 - DoS - Spam - Hosting pirated software - Scanning for reconnaissance - Confidential information theft (spyware) - Keylogging.
- Inspecting Malware Code:
 - To know what a malware does and its internal design, need to look at the source code.
 - Source code is not available to the user.
 - If the executable code of the malware is there, software reverse engineering techniques can be used to get its code in high level programming languages.



- Disassemble & Decompile Process:
- ELF File Format (Executable and Linkable Format)
 - Executable and Linkable Format (ELF) is a file format used for executable files on Unix-like platforms.
 - When running a program, an ELF file is read by the linker and loader to make it runnable from the RAM.
 - The readelf utility program on Linux can be used to read the content of an ELF file and display it.
 - A reverse engineering tool will read an ELF file (and various other executable file formats) and disassemble and decompile it.
- Ghidra → Software Reverse Engineering (SRE) suite of tools developed by NSA's Research directorate.
 - NSA later classified it and turned it into an open source project.
 - Ghidra decompiles an ELF and provides the high-level code. Can be used to debug an executable program.



Approaches to find the identity of a dark web website operator

Application layer info leakage

- **Server Misconfigurations:** Investigators scan the hidden service for misconfigured HTTP headers that might accidentally reveal the server's real "Clear Web" IP address.
- **Error Message Analysis:** Triggering specific errors (e.g., 404 or 500 errors) can sometimes cause the server to leak its internal hostname or local IP address in the response body.
- **Metadata in Hosted Files:** Any files (images, PDFs, or software) hosted on the site should be analyzed for EXIF data or author metadata, which could reveal the creator's real name, software version, or even GPS coordinates.

OSINT and Digital Footprinting

- **Alias/Handle Correlation:** Investigators use Open Source Intelligence (OSINT) to search for the website's name or the admin's pseudonym across clear-web forums like Reddit, GitHub, or StackOverflow.
- **Historical Data:** If the site was ever hosted on the clear web before moving to the Dark Web, historical WHOIS records or the Wayback Machine might reveal the original registrant's details.

Network and SCA

- **Traffic Correlation Attacks:** While Tor encrypts traffic, large-scale investigators (like state actors) may perform timing analysis—correlating when a hidden service is active with the traffic patterns of specific ISP users.
- **Leaked Credentials:** Searching databases of known "Clear Web" data breaches for the admin's common aliases or passwords can lead to their real-world email accounts.

Financial Tracking (Cryptocurrency)

- **Blockchain Analysis:** If the site accepts Bitcoin or other non-privacy coins, investigators track the "money trail" on the public ledger.
- **KYC (Know Your Customer) Off-ramps:** The identity is often revealed when the operator attempts to move funds from their anonymous wallet to a regulated cryptocurrency exchange to cash out.

Approaches to find the identity of a person using a dark web

Browser Fingerprinting and Exploits

- **Standard Fingerprinting:** Investigators can embed specialized scripts on a Dark Web site that collect details about the user's browser, such as screen resolution, installed fonts, and time zones.
- **Tor Browser Vulnerabilities:** If an investigator (or a state actor) controls a Dark Web site, they might use a browser-based exploit (like a Javascript vulnerability) to force the user's computer to send a "ping" to a clear-web server, revealing their real IP address

Client-Side OPSEC Failures

- **Username/Handle Reuse:** Many users use the same pseudonym on the Dark Web as they do on Clear Web platforms like Reddit, Gaming forums, or Twitter. Investigators use OSINT to link these identities.
- **Revealing Personal Info:** Users often accidentally leak personal details in chat rooms or forum posts—mentioning their local weather, specific hobbies, or even uploading photos that contain EXIF metadata (GPS coordinates).
- **PGP Key Analysis:** If a user provides a PGP public key for encrypted communication, investigators search PGP key servers to see if that same key is linked to a real-world email address or name.

Traffic and Timing Analysis

- **ISP Correlation:** By obtaining logs from an Internet Service Provider (ISP), investigators can look for users who connect to the Tor network at the exact same time a specific activity occurs on a Dark Web service.
- **Guard Node Analysis:** If an investigator can identify or control the first node in the Tor circuit (the Entry/Guard node), they can see the user's real IP address, though they still wouldn't know what the user is doing without further correlation.

The "Money Trail"

- **Cryptocurrency Linkage:** If the user makes a purchase, the transaction is recorded on the blockchain.
- **Exchange Identification:** Investigators follow the funds. If the user ever transfers cryptocurrency from a "clean" account (linked to their real bank account via a regulated exchange) to their Dark Web wallet, the anonymity is broken.

Physical and Mobile Forensics

- **Device Seizure:** If a suspect's phone or laptop is seized, forensics tools can find artifacts of Dark Web use, such as the Tor Browser installation, cached thumbnails of visited .onion sites, or saved login credentials.
- **App Synchronization:** Some mobile apps may inadvertently sync data or notifications related to Dark Web activity to clear-web cloud accounts (like iCloud or Google Drive), providing a link to the user's real identity.

If a fixed SIM card is used:

1. Increased Risk of Remote Data Destruction

- **Standard Procedure:** In a normal investigation, one of the first steps to isolate a device is to physically remove the SIM card to disconnect it from the cellular network.
- **Implication:** If the SIM is fixed, this physical isolation is impossible. The investigator must rely entirely on Faraday bags or software settings like Airplane Mode. If the device is locked and cannot be put into Airplane Mode, it remains connected to the network, making it highly vulnerable to a remote wipe command from the suspect.

2. Stronger Evidentiary Attribution

- **Standard Procedure:** Suspects often swap SIM cards to distance themselves from certain communications.
- **Implication:** A fixed SIM creates a permanent logical link between the IMEI (Handset identity) and the IMSI (Subscriber identity). This makes it much easier for a forensic investigator to prove that a specific physical device was responsible for a specific network activity, as the "swapping" defense is removed.

3. Challenges in Data Acquisition

- **Standard Procedure:** Investigators often use specialized SIM card readers to extract contacts, SMS, and location logs directly from the chip without interacting with the phone's OS.
- **Implication:** With a fixed SIM, the investigator is forced to perform a Logical or Physical acquisition through the phone's data port. If the device is physically damaged or has strong screen-lock security, the SIM data may become completely inaccessible, whereas a removable SIM could have been read independently.

4. Impact on Network Provider Records

- **Standard Procedure:** Investigators correlate phone data with Call Detail Records (CDR) from the service provider.
- **Implication:** Because the handset and SIM are a fixed pair, the service provider's logs become much more reliable for forensic mapping. Any location data or call logs provided by the carrier can be attributed to that specific physical device with much higher certainty.

An investigator uses a USB-based tool to acquire the RAM of a live laptop.

- It violates Principle 1 because the acquisition tool itself must be loaded into memory, which overwrites some of the original data.
- It is forensically sound because the investigator can justify the change as necessary to preserve volatile evidence that would otherwise be lost. As long as the process is documented and performed by a competent person (Principle 2), it is admissible.

A suspect uses a rootkit to hide a malicious process from the Windows Task Manager.

- pslist would likely fail because it relies on the same high-level system lists that the rootkit has tampered with.
- psscan should be used. It scans the raw memory for process objects (EPROCESS structures) directly, bypassing the OS's list-based view and revealing processes the malware tried to hide.

To detect such malware, an investigator should follow these systematic steps:

1. Live Memory Acquisition

- The investigator must first acquire a forensic image of the RAM while the computer is still powered on.
- This must be done before shutting down or pulling the plug, as memory-resident malware is volatile and will be lost once power is removed.

2. Identifying Hidden Processes

- Malware often hides from standard OS tools like Task Manager. An investigator should use the psscan command in Volatility.
- Unlike pslist, which follows the linked list of active processes, psscan searches for raw EPROCESS structures in memory, revealing hidden or terminated malicious processes.

3. Detecting Code Injection and Hooks

- Memory-based malware often injects its code into legitimate processes (like explorer.exe or svchost.exe).
- The investigator should use malfind to locate memory regions that have "Execute/Read/Write" (ERW) permissions, which are characteristic of injected malicious code.
- They should also check for API Hooking using the apihooks plugin to see if the malware is intercepting system calls to hide its activity.

4. Network Connection Analysis

- Even if the malware is fileless, it usually needs to communicate with a Command and Control (C2) server.
- Running netscan allows the investigator to identify suspicious active or closed network connections, remote IP addresses, and the specific processes responsible for that traffic.

5. Extracting and Analyzing Artifacts

- Once a suspicious process is identified, the investigator can use procdump or vaddump to extract the malicious code from the RAM for further Static or Dynamic Analysis.
- They can then search for "beacons" or "strings" within the dumped memory that reveal the malware's capabilities.